AI Engineering and Claude Architect · Part IV, Production

AI Production Engineering Exercises

Six Worked Exercises

Fix a real bug, read a trace, calibrate a pipeline, price a cache, plan a migration, and defend a decision.

Module	AI10 of 13
Covers	All 7 lessons
Time	~3 hours

Exercise 1: Fix the spend-cap handling

`SpendCapTest` shows that AI08's `ClaudeModel` maps a spend-cap 429 to `RATE_LIMITED`, marked retryable. The SDK also made three requests.

(a) How should the code tell them apart? **(b)** What should change in `ModelException`, and what should the caller see? **(c)** Can you stop the SDK's two extra requests? **(d)** Also, what about a spend limit you set?

Solution

(a) Per the rate limits page, on the Messages API `error.details.error_code` is `enforced_spend_limit_reached`. Read that from the exception's body. Don't string-match the message. A missing `retry-after` is a hint and not proof, as some rate-limit 429s may lack it too.

(b) Add `SPEND_CAP(false, 503)` to `Kind`. It isn't retryable, and the caller's status shouldn't invite a retry. Log at error level and page someone, as only a person can fix it.

(c) Not by inspecting the body, unless your SDK version exposes a retry hook. I haven't found one in `anthropic-java`, and haven't searched exhaustively. Setting `maxRetries(0)` and retrying in your own layer gives full control, and that fits the "one layer owns retries" rule. Otherwise, accept two wasted calls: a 429 here costs no tokens, so this is a small price, and the main fix is in the mapping.

(d) The page says a limit you set returns a 400 `invalid_request_error`, with a message beginning `You have reached your specified API usage limits`. AI08 maps 400 to `BAD_REQUEST`, which reads as a code bug. The errors page offers no error code for this in what I read, so this case does need a message check. Map it to the same `SPEND_CAP` kind. The docs advise against matching messages generally, so isolate it and cover it with a test.

Exercise 2: Read this trace

A support agent's run, on Sonnet 5.5 ($2 input, $0.10 cache read, $10 output, per million):

Span	ms	Input	Read	Output
Turn 1	1,500	500	12,000	80
`get_customer`	90			
Turn 2	1,800	600	12,000	90
`search_kb` returns 20,000 tokens	300			
Turn 3	6,000	20,500	12,000	350
Turn 4	2,500	300	32,000	120

(a) Compute each turn's cost, and the total. **(b)** What's the biggest problem? **(c)** Propose a fix, and estimate the effect if `search_kb` returned 2,000 tokens.

Solution

(a)

Turn	Input	Read	Output	Total
1	$0.0010	$0.0012	$0.0008	$0.0030
2	$0.0012	$0.0012	$0.0009	$0.0033
3	$0.0410	$0.0012	$0.0035	$0.0457
4	$0.0006	$0.0032	$0.0012	$0.0050
Total				**$0.0570**

(b) `search_kb` returned 20,000 tokens. Turn 3 is 80% of the cost, and it also took the longest. Turn 4 shows that those tokens stay in the history as cached reads, so a longer run would pay for them repeatedly.

(c) Return fewer, better chunks (AI05), or trim to the fields needed (AI06). With 2,000 tokens: turn 3's input is 2,500, costing $0.0050, for a turn total of $0.0097. Turn 4's reads fall to 14,000, costing $0.0014, for a turn total of $0.0032. New total: $0.0030 + $0.0033 + $0.0097 + $0.0032 = **$0.0192**, about 66% lower. This assumes that the answer stays as good, so run the eval.

Exercise 3: Calibrate a pipeline

You have a validation set of 10,000 labelled fields from a contract extractor:

Stated confidence	n	Correct
Below 0.80	1,000	620
0.80–0.90	2,000	1,700
0.90–0.95	3,000	2,880
0.95 and above	4,000	3,960

(a) Compute real accuracy per band. Is the model over- or under-confident? **(b)** Reviewers can handle 30% of fields. Which threshold, and what accuracy results in the accepted share? **(c)** Why isn't that enough to proceed?

Solution

(a) 62.0%, 85.0%, 96.0% and 99.0%. Below 0.80, it's roughly in line or uncertain. At 0.80–0.90, it's about right. Above that, 96.0% sits above the 0.90–0.95 band, and 99.0% is at the top of its band. So it's not badly calibrated here, though the band edges are coarse. Check with narrower bands.

(b) Review the bottom two bands, which is exactly 3,000 fields, or 30%. Accepted: 7,000 fields, with 6,840 correct, which is **97.7%**. Of 3,000 reviewed, 680 contained errors, so reviewers find 680 of the 840 total errors, about 81%. The remaining 160 errors go through.

(c) Three gaps. It's an aggregate, so one document type or field may still be poor, so break it down first (TS 5.5). A threshold chosen on this data needs confirming on separate test data. And after going live, take a stratified audit sample of accepted fields, as otherwise those 160 errors per 10,000 will never be seen. Also route contradictory documents to people, whatever the score.

Exercise 4: Price a cache

A public FAQ bot takes 3,000,000 questions a month on Haiku 5.5. Each has 1,500 input tokens and 150 output tokens. Estimates, from a labelled sample: semantic cache at a chosen threshold would hit 35%, with 0.2% of hits wrong. A wrong answer costs $0.50 on average (follow-up tickets). Embedding and lookup cost $0.00002 per question.

(a) Compute cost without a cache. **(b)** Compute net effect. **(c)** Would your answer change for a bank's bot where a wrong answer costs $50?

Solution

(a) Each call: 1,500 × $0.10/M + 150 × $0.50/M = $0.00015 + $0.000075 = $0.000225. Monthly: **$675**.

(b) Hits: 1,050,000. Savings: 1,050,000 × $0.000225 = $236.25. Lookup cost: 3,000,000 × $0.00002 = $60. Wrong answers: 2,100, at $0.50 each = $1,050. Net: 236.25 − 60 − 1,050 = **−$873.75 a month**. The cache loses money, because Haiku is so cheap. This is a good reminder to do the arithmetic before building it, even for an FAQ. Latency might still justify it, in which case you're paying for speed, so make that the stated reason.

(c) At $50, wrong answers cost $105,000 a month. No. Use an exact cache, or intents with reviewed answers, as in L5.

Exercise 5: Plan a migration

It's 5 November 2026. Your document assistant runs on `claude-sonnet-4-5-20250929`, retiring on 30 November. It uses `temperature: 0.2`, and forced `tool_choice`, with one fallback on Bedrock. It handles 400 labelled outcomes a week.

(a) Which breaking changes do you expect? **(b)** Lay out the 25 days. **(c)** What can the canary detect? **(d)** What's your plan if evals fail on the 20th?

Solution

(a) From the docs I read: `temperature` returns a 400 on API version `2026-10-21`, and on earlier versions when non-default on recent models. Forced `tool_choice` returns a 400 on Sonnet 5.5. So use `auto` with strict tools, or structured outputs. Thinking defaults differ. Output style may change, so recalibrate thresholds. And the Bedrock fallback has its own retirement schedule and supported features, which need checking.

(b) Days 1–2: inventory, using the Console's usage export for every key on this model, and read the migration guide. Days 3–7: fix parameters, run the eval suite, compare by segment, and recalibrate. Days 8–12: shadow on real traffic, with no write tools. Days 13–20: canary by session, then ramp. Days 21–25: margin. Don't plan to finish on the last day.

(c) Very little. At 400 outcomes a week, a 50/50 canary gets about 200 per arm a week. Detecting 5 points from 95% needs about 431 per arm, so about two weeks. Smaller drops can't be seen. So the offline eval carries the weight of quality, and the canary mainly catches operational problems.

(d) Find out which segments fail. Options: fix the prompt, try `claude-opus-5-5` for the failing segment, or send that segment to people temporarily. Staying on 4.5 past the date isn't an option on the Claude API, as requests will fail. Contact Anthropic support, as the docs suggest for help, but don't count on an extension. And record the lesson: start on the day that notice arrives.

Exercise 6: Defend a decision

The decision: a SaaS company with six teams, on one provider, must decide: build an LLM gateway, adopt an existing one, or keep a shared client library with workspaces. Write your recommendation, with its risks.

Model answer

I'd begin with a shared library plus workspaces, and set clear triggers for moving to a gateway.

What each need actually requires. Attribution: workspaces and the Usage and Cost Admin API provide this by key and workspace. Per-team limits: workspaces give rate and spend caps. Model migration: logical names in the library. Retries and classification: one tested implementation in the library, which is where L1's spend-cap fix would go. That covers most of what six teams need, with no new critical component on the request path.

What it doesn't do. Keys still live in services, so revocation and rotation take more work. Per-tenant fairness inside a team stays with that team. It needs a library per language: if there are three languages, that weakens the case. And it can't force compliance, so someone can call the API directly.

Why not build now. A gateway becomes critical to every feature, needs staffing, and must keep up with API changes. I measured a local relay at a few milliseconds, and that's the easy part. The budgets, failure behaviour, and pass-through are the real work, and L7's design shows how much there is.

Why not buy now. It could be right. It would handle key removal fastest. But it sees all prompts, needs a security review, and I haven't evaluated any product, so I can't claim one fits. I'd give that a short evaluation if a trigger below fires.

Triggers to move. A security requirement to remove provider keys from services. Needing a second provider or platform with shared routing. Needing per-tenant budgets as a product feature, across teams. More than two languages. Or a key leak.

Risks I'm accepting. Teams may drift from the library, so CI should check the version. Spend attribution is only as good as workspace discipline. And a later migration to a gateway has a cost, kept small because clients only need a new base URL if the gateway uses the provider's format.

What would change my mind. A compliance requirement arriving next quarter would make me go straight to a gateway, and adopt rather than build.